

Project Proposal

CMSC 126 - Web Engineering

Team	CATcha
Members	<i>Max Lennon Limpag</i> <i>John Carlo Sandro</i> <i>Selena Therese Malig</i> <i>Isabella Nicole Recilla</i> <i>Ishah Nicholei Bautista</i>

Project Name	StudyStack
Project Description	An academic platform where student questions surface gaps in understanding, and the community works together to resolve those gaps. Instead of simply hosting questions and tutorials, StudyStack prioritizes content based on community demand. Questions rise in visibility as more users express the need for answers, and when answers are insufficient, the platform encourages the creation of tutorials to address deeper learning needs.
Core Concept	Core Concept: Demand-Driven Learning Loop StudyStack operates on a central lifecycle: Question rises → Community signals demand → Answers attempt resolution → Demand persists → Tutorial created → Tutorial vetted → Question resolved (or cycle repeats)
Primary Features	
Primary Features 1. Demand Score System Each question has a dynamic <i>demand score</i> that reflects how in-demand the question is and the urgency for answers. Demand increases through likes and negative feedback on answers, and decreases as helpful answers or tutorials are provided. 2. Rising Board The homepage displays a live feed of questions ranked by demand. Questions rise as demand increases and are slowly removed once resolved by answers or tutorials.	

3. Q&A Discussion System

Students can post questions and provide answers. Answers are evaluated through community feedback ("*Was this helpful?*"), which directly affects both answer credibility and question demand.

4. Tutorial Creation Linked to Questions

Tutorials are created in response to high-demand questions and are directly tied to resolving those questions.

5. Tutorial Vetting System

Tutorials are evaluated by the community. If deemed effective, they resolve the associated question; otherwise, demand persists and the cycle continues.

6. Commenting System

Users can engage in discussions through comments on questions, answers, and tutorials.

7. Real-time Interaction

Users receive live updates for new answers, comments, and interactions through WebSockets.

8. Search and Discovery

Users can search for content across questions, tutorials, and profiles.

9. Content Moderation System

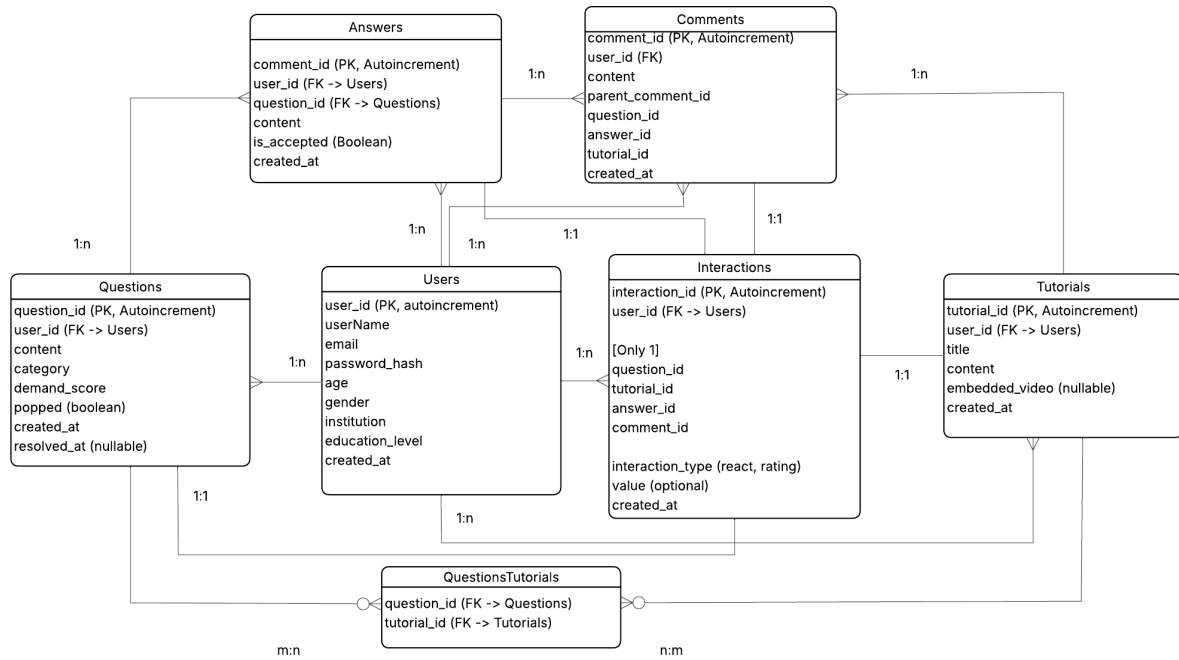
Admins can manage flagged or inappropriate content.

10. Offline Support

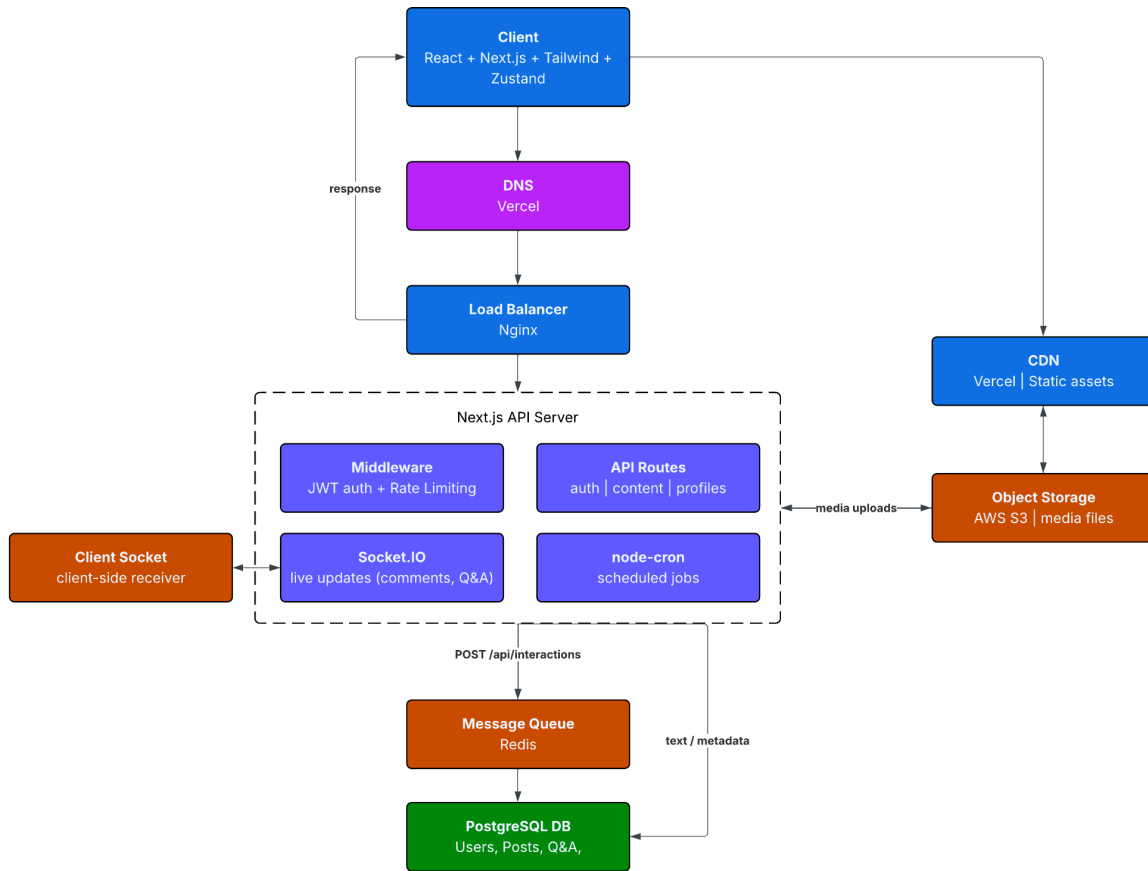
Previously accessed content remains available through service worker caching.

Tech Stack	
Frontend	React + Tailwind CSS
Full-stack Framework	Next.js for frontend rendering and API routes
Database	PostgreSQL
Websocket	Socket.IO
Cron	Node Cron
Authentication	JSON Web Tokens (JWT) auth
Hosting & CDN	Vercel

Database Schema



System Architecture



Data Flow and Client-Server Connection	
<i>Outbound Client (Client to Server)</i>	
Text/Metadata	HTTP POST/PUT requests (questions, answers, tutorials, interactions)
Media	Cloud Storage Bucket (AWS S3), URL and Metadata written to DB
<i>Inbound Data (Server to Client)</i>	
Retrieval	HTTP GET requests to fetch paginated content (questions, tutorials, answers)
Caching	Static or infrequently updated data is stored in local client cache or state management stores
<i>Processing Logic</i>	
Demand Processing	User interactions (e.g., "Was this helpful?") update demand scores and answer credibility
Tutorials	High bandwidth HTTP / Content Delivery Network (CDN) routing
Communication	Low latency HTTP, Websocket for real-time updates
Profiles	HTTP GET requests for user data and aggregated statistics

Data Payloads and Endpoints	
<i>Data Payloads</i>	
<i>Client To Server</i>	
User Register	<pre> { "userName": "plain_student", "userID": "USER-9921", "email": "student@upcebu.edu.ph", "password": "my_plain_text_password", "profile": { "age": 20, "gender": "Male", "institution": "University of the Philippines Cebu", "educationLevel": "Undergraduate" } } </pre>

User Interactions	<pre>{ "actorId": "USER-9921", "data": { "interactionType": "like", "targetType": "tutorial", "targetId": "TUT-88245", "value": helpful } }</pre>
User Question	<pre>{ "authorId": "USER-4056", "data": { "category": "Computer Science", "questionContent": "How do you implement the Typestate Pattern in Rust to enforce strict transitions at compile time?" } }</pre>
User Tutorial	<pre>{ "authorId": "USER-1092", "data": { "targetQuestionId": "QUES-7731", "title": "A Guide to Typestate in Rust", "content": "Here is a complete markdown breakdown of the pattern...", "embeddedVideoUrl": "https://youtube.com/watch?v=..." } }</pre>
User Comment	<pre>{ "authorId": "USER-4056", "data": { "commentType": "answer", "targetType": "question", "targetId": "QUES-7731", "content": "You can use phantom data to track the state..." } }</pre>
Server To Client	
User Information	<pre>{ "userName": "",</pre>

	<pre>"userId": "", "profile": { "gender": , "age": , "email": , "institution": , "education_level": , } "userStats": { "total_likes": , "total_answers": , "total_questions": , "helpfuls": , } }</pre>
Content Detail	<pre>{ "contentId": "POST-88245", "contentType": "video", "author": { "id": "USER-1092", "name": "jcsandro99", "reputation": 450 } "title": "Understanding Core Concepts", "body": "Markdown or HTML string representing text content...", "mediaUrl": "https://cdn.example.com/assets/video123.mp4", "statistics": { "likes": 1250, "dislikes": 12, "downloads": 45 }, "metadata": { "section": "tutorial", "timestamp": "2026-04-11T13:51:00Z", "status": "published", "demandScore": 42 } }</pre>
API Endpoints	
POST /api/auth/register	Handles user creation and JWT distribution

POST /api/auth/login	Handles user logins and JWT distribution
POST /api/auth/logout	Invalidates sessions, logging the user off
GET /api/users/:userName	Fetch a user's profile information
PUT /api/users/:userName	Update the user's basic personal information
DELETE /api/users/:userName	Delete a user's account. The process is verified by user_id, JWT, and the user inputting the current password to confirm deletion.
GET /api/content	Fetches paginated arrays of content objects (filtered by section: tutorial or discussion).
GET /api/content/:content_id	Fetches a paginated list of questions which is filterable by <i>category</i> .
GET /api/questions	Fetches a paginated list of questions which can be filterable by <i>category</i> .
GET /api/questions/:questionId	Fetches a single question by ID including answers.
POST /api/questions	Create a question and update the user's total_question in UserStats. The ownership of creation is verified by user_id and JWT.
PUT /api/questions/:questionId	Edit a question, with ownership verified by user_ID and JWT.
PATCH /api/questions/:questionId	Mark a question as resolved and set a date when it is resolved. The ownership is verified by user_ID and JWT.
DELETE /api/questions/:questionId	Deletes a question with ownership of a question verified by user_ID and JWT.
GET /api/questions/:questionId/answers	Fetches answers that are under a specific question
GET /api/answers/:answerId	Fetches a specific answer that's under a specific question.
POST /api/questions/:questionId/answers	Creates an answer under a specific question. Ownership is verified by userID and JWT.
PUT /api/answers/:answerId	Edit an answer made by the user. This process verifies the user by JWT and user_id.

DELETE /api/answers/:answerId	Delete an answer made by the user. This process verifies the user by JWT and user_id.
GET /api/questions/:questionId/answers/:answerId/comments	Fetches comments that are answer type under a specific question
GET /api/comments/:commentId	Fetches a specific comment.
POST /api/answers/:answerId/comments	Create an answer for a question and update total_answers in UserStats. The ownership of the answer is verified by user_id and JWT
PUT /api/comments/:commentId	Edit an answer or upvote a comment with ownership verified by user_ID and JWT.
DELETE /api/comments/:commentId	Deletes an answer with ownership of a question verified by user_id and JWT. Afterwards, update the total_answers in UserStats.
POST /api/interactions	Add an interaction
DELETE /api/interactions/:interactionId	Remove/undo an interaction
GET /api/interactions?targetId=&targetType=	Pass target_id and target_type to get specified type of content that the user interacted with
GET /api/tutorial	Fetches a paginated list of tutorials that can be filtered by category.
GET /api/tutorial/:tutorialId	Fetches a single tutorial by ID, including answers.
POST /api/tutorial	Create a tutorial. The ownership of the tutorial is verified by user_id and JWT
PUT /api/tutorial/:tutorialId	Edit a tutorial, with ownership verified by user_ID and JWT.
PUT /api/tutorial/:tutorialId/upload	Create a tutorial. The ownership of the tutorial is verified by user_id and JWT
DELETE /api/tutorial/:tutorialId	Deletes a tutorial with ownership of tutorial verified by user_id and JWT.

<i>Cron Jobs</i>	
Recalculate feed based on demand scores	0 * * * *
User rating and engagement update	0 * * * *
Cleanup (delete spam/reported content, remove soft-deleted posts)	0 3 * * *

Low Fidelity UI

Login Page

StudyStack

Welcome back! Please sign in.

Email

max.lennon@school.edu

Password

Sign in

[Forgot password?](#)

Register Page

Join StudyStack

Create your academic profile

Full Name

Max Lennon Limpag

Email

max.lennon@school.edu

Institution & Education Level

UP Cebu

Undergrad

Password

Create account

By signing up, you agree to our terms.

Home Feed

StudyStack

☐ Feed

☐ Trending

☐ Bookmarks

Topics

CMSC

Math

Physics

Others

Rising Board

Tutorials

Ask Question

Rep: 1.2k JD

Search questions, tutorials, users...

Search

RISING BOARD (SORTED BY DEMAND)

98

How to implement binary search in Java?

Existing answers are good but we need a visual tutorial for the off-by-one errors...

Open 12 answers 2 hrs ago CMSC 123

5 comments Bookmark

42

Confused about pointers in C, dereferencing?

Pointers are confusing. When do I use * and when do I use &?

Closed 3 answers 1 day ago CMSC 121

3 comments Bookmark

15

Intro to Normalization: 1NF to 3NF explained

Tutorial Linked CMSC 126 5 hrs ago

15 8 View Tutorial

Tutorials Page

StudyStack

Rising Board

Tutorials

Ask Question

Rep: 1.2k JD

Search tutorials...

All

Video

Text

CMSC

Math

Sorting Algorithms Visualized

Author: Selena M. Video

89 12 Bookmark

Understanding ERD Relationships

Author: John Carlo S. Text

56 9 Bookmark

Ask Questions

StudyStack

Rising BoardTutorialsAsk Question

Rep: 1.2kJD

Ask a question

What's on your mind? Be specific...

+ Add topicPost

AllUnansweredHigh Demand

98

How to implement binary search in Java?

Create Tutorial

KL

Kim L.

CMSC 123

Asked 2 hrs ago

I keep getting an off-by-one error. Specifically, should it be `while(low < high)` or `while(low <= high)`?

2 ANSWERS

SORT BY: CREDIBILITY

▲

Credibility: 94%

12

Use `mid = low + (high - low) / 2` to prevent overflow. If you use `mid - 1` and `mid + 1`, then `low <= high` is correct.

▼

Answered by Andrea R. 50 mins ago

This is the standard approach in most libraries. — Selena M.

Add a comment...

Helpful?👍👎

▲

Credibility: 12%

👍👎

Profiles

StudyStack

HomeTutorialsQ&A

JD

JD

Ishah Nicholei Bautista

BSCS — 3rd year @ UP Cebu

Edit profile

14

Questions

38

Answers

221

Likes Given

4.8

Rating

High

Engagement

Tutorials

Questions asked

Answers given

Understanding ERD Relationships

Tutorial3 days ago

▲ 56🗨 9EditDelete

Sorting Algorithms Explained Simply

Tutorial1 week ago

▲ 89🗨 12EditDelete